Coding Bootcamp Database

Individual Database Project: Final Report

Bolaji Parchment

1. Mini World Selection – Coding Bootcamp

1.1 Project Overview

This database project represents a 12-week coding bootcamp. It runs a day cohort and a night cohort, allowing only 50 students to join per cohort. The database will track applications, enrollment, payment, attendance, assignments, submissions, tutoring and professional development coaching. There are constraints and triggers in place to enforce the rules such as cohort capacity, failure due to absences, and shift matching between students and cohort, cohort and cohort unit, and so on and so forth.

1.2 Research

To begin this project, I researched Hack Reactor. Hack Reactor offers a 16-week curriculum that covers 9 units as well as career coaching. Their goal is to prepare students for the modern workforce demands by providing fundamental tools, including AI. The weekly schedule is Monday-Thursday from 9am-5pm and every other Friday from 9am-5pm. They have experienced many success stories, one particularly who I had the chance to interview.

1.3 Interview

Cise Babatasi is a former Hack Reactor student who is now a Software Engineer at Thryv and I asked her some questions about her learning experience, attendance, and outcomes.

- **How was the curriculum organized (modules or week by week)?**

  "The program was a 19-week intensive bootcamp divided into three main phases.

  During the first six weeks, we focused on learning technical concepts through short, usually two-day coding sprints. We covered JavaScript, Python, data structures, algorithms, React, Node.js, databases, and full-stack development.

  The second half focused more on larger team projects, applying what we'd learned, and preparing for real software engineering jobs."

- **What kind of assignments did you have (projects, quizzes, code challenges)?**

“It was very hands-on. We had daily coding challenges, pair-programming exercises, technical assessments, and group projects.

Most assignments involved writing actual code rather than taking traditional quizzes.

We also had a major technical assessment halfway through the program and larger capstone projects toward the end, where we worked in teams to build applications.”

- **How did you submit work, and were you allowed to resubmit?**

“We primarily used GitHub to submit coding assignments and projects. We also had an internal learning platform where we accessed instructions and tracked coursework.

For regular exercises, we could continue improving our code and get feedback from instructors.

I don’t remember there being a universal resubmission policy for every assignment. Formal technical assessments were handled differently from regular practice exercises.”

- **How was attendance taken, and what would happen if you missed class?**

“Attendance was pretty strict because the program moved very quickly.

Staff tracked attendance at the beginning and end of the day, including absences and late arrivals. There was an attendance-point system, and missing too much class could affect your ability to continue the program.

If someone needed to miss class, they were expected to communicate with the instructional team.”

- **Did you have more than one instructor?**

“Yes, the program had multiple people supporting students, including instructors, technical mentors, and teaching assistants.

Different instructors could lead lectures or technical sessions, while mentors and teaching assistants helped with coding exercises, debugging, and answering questions.

I liked that structure because we could get support from different people instead of depending on only one instructor."

- **What could you see about your own progress (grades, attendance, completion)?**

"We had access to our coursework through the learning platform, and we received feedback on coding assignments and technical assessments.

There were regular self-assessments and instructor evaluations, so we could understand where we were doing well and where we needed improvement.

Attendance was also tracked by the school, although I don't specifically remember whether we could see a complete attendance history ourselves.

Progress feedback was more focused on technical skills and completing the program than a traditional school-style GPA."

- **What information would you have wanted the bootcamp to track that it didn't?**

"I think it would have been really useful to have one dashboard showing our overall progress.

For example, I'd have liked to see which assignments I'd completed, my technical strengths and weaknesses, my assessment results over time, and my attendance in one place.

I'd also have liked clearer information about how my technical progress compared with the skills expected for job interviews.

That would have helped me identify where to focus my time, especially because the program moved so quickly."

2. Database Requirements

After researching Hack Reactor and interviewing Cise, I was able to determine the requirements for my database.

2.1 Applying for bootcamp

When applying for the coding bootcamp, there is a mandatory entry exam that must be passed before approval into the bootcamp. Tuition cost is $10,000 and a payment plan is available upon

approval. The payment plan is $2,000 up front, $4,000 by week 6, and $4,000 by week 12. Once a cohort is full, the application closes until the 12 weeks are over.

2.2 Cohort Schedule

The weekly schedule is Monday-Friday from 8:00pm-4:00pm for the day shift and 2:00pm-10:00pm for the night shift. I decided to implement a tutoring session that is available to all students every Saturday from 12:00pm-2:00pm; the sessions are separate for each cohort so that no professor will teach more than 50 students at a time.

Day Shift	Night Shift	Block	Tutoring (Saturday)
8:00am – 8:20am	2:00pm – 2:20pm	Do-Now	12:00pm – 2:00pm
8:20am – 11:00am	2:20pm – 5:00pm	Lecture 1	
11:00am – 11:30am	5:00pm – 5:30pm	Knowledge Check	
11:30am – 12:30pm	5:30pm – 6:30pm	Lab 1	
12:30pm – 1:15pm	6:30pm – 7:15pm	Lunch/Dinner	
1:15pm – 2:45pm	7:15pm – 8:45pm	Lecture 2	
2:45pm – 3:45pm	8:45pm – 9:45pm	Lab 2	
3:45pm – 4:00pm	9:45pm – 10:00pm	Closeouts	

2.3 Cohort Units

The units offered are Python foundations, HTML + CSS Basics, Javascript foundations, Database Management, Introduction to Windows Terminal and Professional Development. Each

unit will be covered over the course of 2 weeks. Tutoring sessions cover the unit being taught that week.

2.4 Assignments

Homework will be given every 2 days and quizzes will be given every 2 weeks. There will be a midpoint assessment and a final project that can be added to the students' resume to help with the job hunt. Labs happen every day during cohort hours, so students are practicing and improving their code daily.

2.5 Attendance

Attendance is mandatory and having more than 4 unexcused absences will result in failure of the bootcamp. It is important that students are responsible, so they take all the knowledge and fundamentals they need away from the bootcamp.

2.6 Professors

There are five professors per cohort. Each professor will teach exactly one unit. Professors also lead tutoring sessions and will rotate each Saturday.

2.7 Professional Development Coaching

There is a professional development coach for each cohort. Each student is expected to meet with the coach once per month. The last 2 weeks of the program will be dedicated to professional development and workplace etiquette so that students know exactly what to bring to interviews and to work when they land a job.

3. Entity-Relationship Diagrams

3.1 Entities and Attributes

Entities	Attributes
student	student_id, first_name, last_name, email
professor	professor_id, first_name, last_name, email, shift
coach	coach_id, first_name, last_name, email, shift

application	app_id, student_id, cohort_id, app_date, approval_status, exam_score, exam_result
enrollment	enrollment_id, student_id, cohort_id, enrollment_date, status, pay_option, final_result
payment	payment_id, student_id, due_date, installment_no, amount_due, pay_date, amount_paid, pay_method
attendance	attendance_id, student_id, attendance_date, status, excused_absence
cohort	cohort_id, coach_id, shift, start_date, end_date, max_students,
unit	unit_id, name, sequence_no, duration_weeks
cohort_unit	cohort_id, unit_id, professor_id, start_date, end_date
assignment	assignment_id, cohort_id, unit_id, professor_id, assigned_date, due_date, max_points, type, title
submission	student_id, assignment_id, submitted_at, score
coaching_meeting	meeting_id, student_id, coach_id, meeting_date, start_time
tutoring_session	tutoring_id, cohort_id, unit_id, professor_id, session_date, start_time, end_time,
tutoring_attendace	student_id, tutoring_id

3.2 Relationships and Cardinality

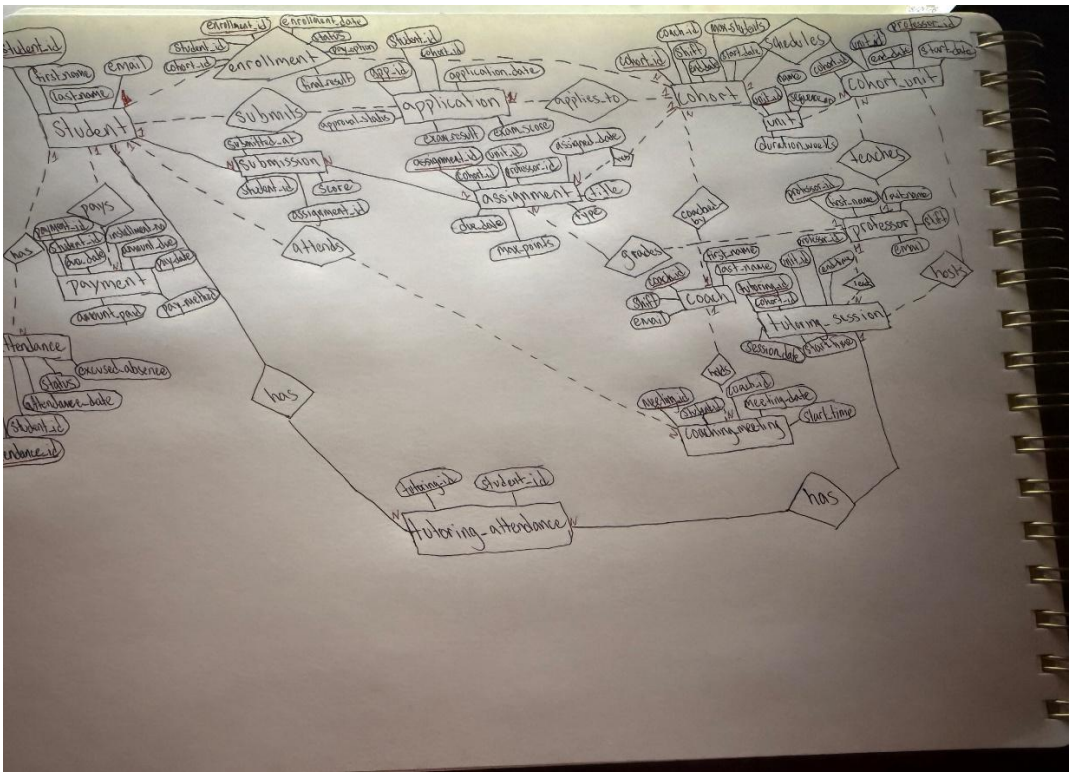
- **Student ↔ pays ↔ Payment: 1:N**
    - One student can make zero or many payments over time
    - Every payment belongs to only one student
- **Student ↔ has ↔ Attendance: 1:N**
    - One student has zero to many daily attendance records
    - Each attendance record references one specific student
- **Student ↔ submits ↔ Submission: 1:N**
    - A student submits zero or many submissions
    - Each submission is from exactly one student

- **Student ↔ Enrollment ↔ Cohort: 1:N**
    - One cohort can have many students enrolled in it
    - Each student can belong to only one cohort at a time
- **Application ↔ applies_to ↔ Cohort: N:1**
    - Many applications can target a single cohort
    - Each application is submitted for one specific cohort
- **Cohort ↔ schedules ↔ Cohort_unit: 1:N**
    - One cohort schedules one or multiple cohort units
    - A specific cohort unit entry belongs to one cohort
- **Professor ↔ teaches ↔ Cohort_unit: 1:N**
    - A professor can teach zero or many cohort units
    - Each cohort unit is taught by exactly 1 professor
- **Professor ↔ hosts ↔ Tutoring_session: 1:N**
    - A professor hosts zero on many tutoring sessions
    - Each tutoring session is hosted by one professor
- **Cohort ↔ coached_by ↔ Coach: N:1**
    - Each cohort is assigned one primary coach
    - A coach oversees zero or multiple cohorts
- **Professor ↔ grades ↔ Submission: 1:N**
    - A professor grades zero to many student submissions
    - Each submission is graded by one professor
- **Coach ↔ holds ↔ Coaching_meeting: 1:N**
    - A coach holds zero or many 1-on-1 coaching meetings
    - Each meeting is led by one coach
- **Student ↔ attends ↔ Coaching_meeting: 1:N**
    - A student attends zero or many 1-on-1 coaching meetings
    - Each scheduled meeting is held for one student

- **Student ↔ has ↔ Tutoring_attendance: 1:N**
    - A student can attend many tutoring sessions
    - Each tutoring attendance record belongs to one student
- **Tutoring_session ↔ has ↔ Tutoring_attendance: 1:N**
    - A tutoring session can have many tutoring attendance records
    - Each tutoring attendance record is linked to one specific tutoring session

3.3 Hand-Drawn Chen Style Diagram

Figure 1: Chen ER diagram of the coding bootcamp database



4. UML (EER) Diagram

4.1 Creating Tables

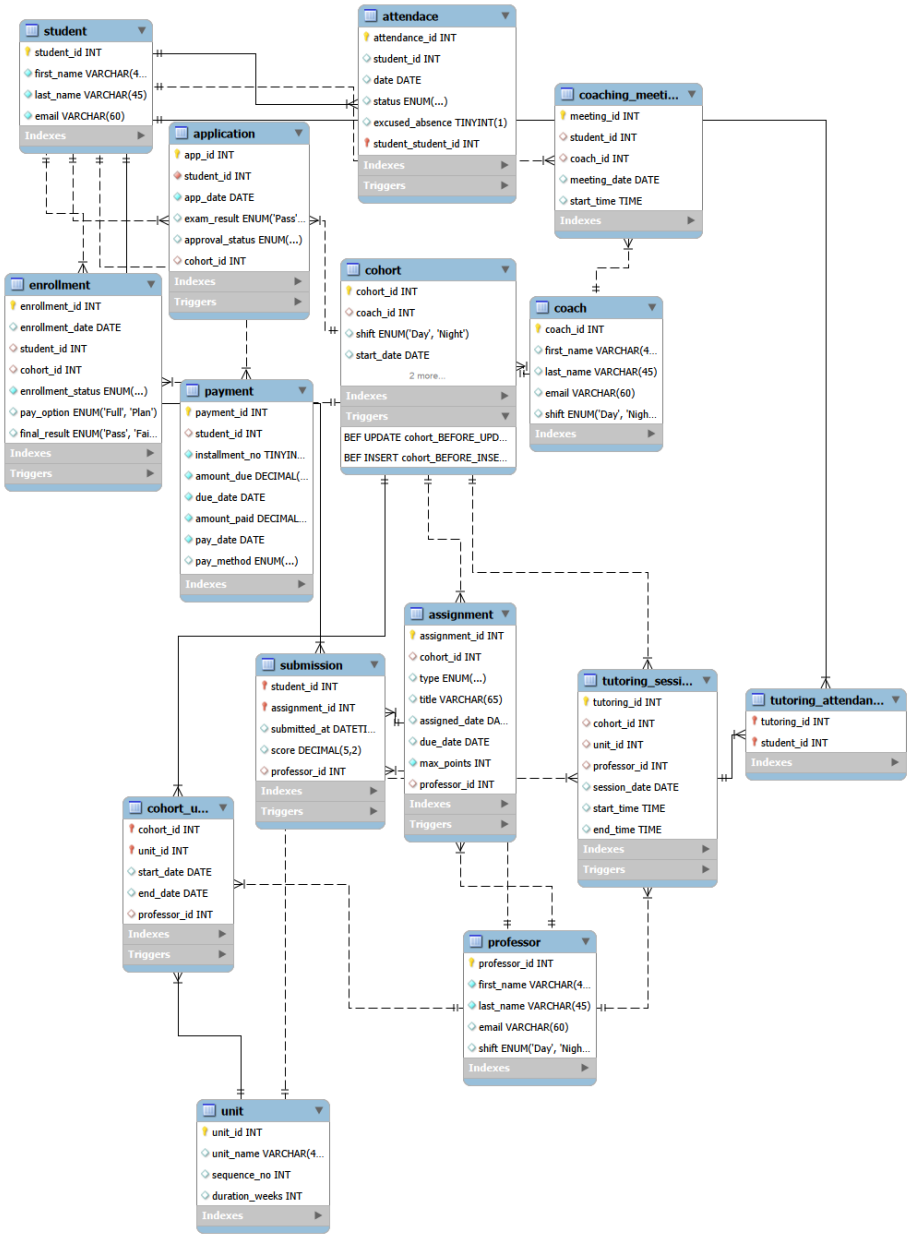
The Chen Diagram helped me create the tables using these rules:

- Each entity became a table, with it's key attribute as the primary key
- Each one-to-many relationship became a foreign key on the "many" side. For example, each application stores the student_id and cohort_id it belongs to.

- Each many-to-many relationship became a table of its own with a combined primary key. For example:
  - The submission table holds the primary key of the student table and assignment table.
  - The tutoring_attendance table holds the primary key of the tutoring_session table and student table.

4.2 EER Diagram

Figure 2: EER Diagram of the coding bootcamp database



5. Database Implementation

5.1 Environment and Tools

- MySQL Server 8.0
- MySQL Workbench; used to write and run SQL scripts and design the EER diagram
- Github; used to store the script, model, and this report

5.2 Creating the Schema

A primary key is the unique identifier of a table. A table can only have one primary key.

A foreign key can only point to a table that already exists so, the tables were created from independent tables to dependent tables. The dependent tables are depending on the primary key from other tables:

1. student
2. coach
3. professor
4. unit
5. cohort (needs coach_id)
6. application (needs student_id and cohort_id)
7. cohort_unit (needs cohort_id, unit_id, and professor_id)
8. enrollment (needs student_id and cohort_id)
9. assignment (needs professor_id, unit_id, cohort_id)
10. submission (needs student_id and assignment_id)
11. attendance (needs student_id)
12. payment (needs student_id)
13. tutoring_session (needs unit_id and cohort_id)
14. coaching_meeting (needs student_id and coach_id)
15. tutoring_attendance (need tutoring_id and student_id)

Constraints are used to enforce the database rules, data integrity and automate actions within the database. The database uses these kinds of constraints:

Constraints	Purpose	Example
PRIMARY KEY	Identifies each row uniquely	student (student_id)
FOREIGN KEY	A value must exist in another table	cohort_unit (unit_id – comes from the unit table)
UNIQUE	Prevents duplicates	application (student_id, cohort_id)
NOT NULL	A value is required	enrollment_date
DEFAULT	Fills a value when none is given	max_students defaults to 50
ENUM	Limits a column to listed values	shift ENUM('Day','Night')
CHECK	Tests a rule within one row	end_date > start_date

Foreign Keys are named with the pattern fk_<table that holds the key>_<table it points to>, for example fk_cohort_unit_unit. On the table it looks like:

```
CONSTRAINT fk_cohort_unit_unit
FOREIGN KEY (unit_id) REFERENCES unit (unit_id),
```

This independent table shows a simple pattern, a generated key (student_id) and necessary columns (first name, last name, and email):

```
CREATE TABLE student (
student_id INT NOT NULL AUTO_INCREMENT,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) NOT NULL,
PRIMARY KEY (student_id),
UNIQUE (email)
);
```

This dependent tables shows more CONSTRAINTS:

```
CREATE TABLE application (
app_id INT NOT NULL AUTO_INCREMENT,
student_id INT NOT NULL,
cohort_id INT NOT NULL,
application_date DATE NOT NULL,
exam_score DECIMAL(5,2) NOT NULL,
exam_result ENUM('Pass','Fail'),
approval_status ENUM('Pending','Approved','Rejected') NOT NULL DEFAULT 'Pending',
PRIMARY KEY (app_id),
UNIQUE (student_id, cohort_id),
CONSTRAINT fk_application_student
FOREIGN KEY (student_id) REFERENCES student (student_id),
CONSTRAINT fk_application_cohort
FOREIGN KEY (cohort_id) REFERENCES cohort (cohort_id),
CHECK (exam_score BETWEEN 0 AND 100),
CHECK (approval_status < > 'Approved' OR exam_result <=> 'Pass')
);
```

5.3 Triggers

Rules that need another row or table, such as counting approved applications or comparing a professor's shift with a cohort's shit are enforced with triggers. Each trigger raises an error with SIGNAL SQLSTATE '45000', which cancels the insert or update and returns a message.

Table	Timing	Rule enforced
cohort	BEFORE INSERT, BEFORE UPDATE	The coach's shift matches the cohort's shift
application	BEFORE UPDATE	Approved applications cannot exceed the cohorts max_students
enrollment	BEFORE INSERT	A student can only enroll with an 'Approved' application for that cohort
attendance	AFTER INSERT, AFTER UPDATE	Four unexcused absences set the student's final_result to 'Fail'
cohort_unit	BEFORE INSERT, BEFORE UPDATE	Unit dates fall inside the cohort's dates; the professor's shift matches the cohort's shift
submission	BEFORE INSERT, BEFORE UPDATE	A score cannot exceed the assignment's max_points
assignment	BEFORE INSERT, BEFORE UPDATE	Assigned and due dates fall inside the cohort's dates
tutoring_session	BEFORE INSERT, BEFORE UPDATE	The sessions is on a Saturday inside the unit's weeks

MySQL does not allow one trigger to cover both the insert and update table so each BEFORE trigger is created twice with the same statement, once for INSERT and once for UPDATE. Here are some examples:

Coach shift matches cohort shift (cohort)

The trigger looks up the shift of the cohort's coach and blocks the row if it is different from the cohort's shift:

```
BEGIN

DECLARE coach_shift VARCHAR(10);
```

```
SELECT shift INTO coach_shift
FROM coach
WHERE coach_id=NEW.coach_id;
IF coach_shift<>NEW.shift THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT='Coach shift must match the cohort shift';
END IF;
END
```

Cohort capacity (application)

When an application changes to ‘Approved’, the trigger counts the approved applications in that cohort and blocks the change if the cohort is full:

```
BEGIN
DECLARE cap INT;
  DECLARE intake INT;
  IF NEW.approval_status = 'Approved' AND OLD.approval_status <> 'Approved' THEN
    SELECT max_students INTO cap
      FROM cohort
     WHERE cohort_id = NEW.cohort_id;
    SELECT COUNT(*) INTO intake
      FROM application
     WHERE cohort_id = NEW.cohort_id
       AND approval_status = 'Approved';
    IF intake >= cap THEN
      SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cohort is full';
    END IF;
  END IF;
END
```

5.3 Testing the triggers

To test the triggers, I entered statements that would break the rule and followed up with ones that follow the rule. For example:

Trigger	Statement that breaks rule	Expected result
Coach shift	Day cohort with a Night coach	Error: Coach shift must match the cohort shift
Score limit	Insert a score above the assignments max_points	Error: Score cannot be greater than maximum points allowed

Coach shift →

Broken statement:

```
INSERT INTO cohort (coach_id, shift, start_date, end_date, max_students)
VALUES (20, 'Day', '2027-02-01', '2027-04-23', 50);
```

Fixed statement:

```
INSERT INTO cohort (coach_id, shift, start_date, end_date, max_students)
VALUES (20, 'Night', '2027-02-01', '2027-04-23', 50);
```

Score limit →

Broken statement:

```
INSERT INTO submission (student_id, assignment_id, submitted_at, score)
VALUES
(103, 1, '2026-11-06 08:35:00', 54.00),
(104, 2, '2026-11-06 23:58:00', 65.00);
```

Fixed statement:

```
INSERT INTO submission (student_id, assignment_id, submitted_at, score)
VALUES
(103, 1, '2026-11-06 08:35:00', 60.00),
(104, 2, '2026-11-06 23:58:00', 58.00);
```

6. Conclusion

This database models the full journey of a coding bootcamp student from application and approval through enrollment, attendance, assignments, submissions, payments, tutoring and professional development coaching. Constraints and triggers keep the data valid and enforce the bootcamp's main rules such as the 50-student cohort limit, shift matching, and attendance.

6.1 Repository Contents

- README.md: project description
- Final Report: this report
- coding_bootcamp.mwb: the MySQL Workbench model, including EER Diagram
- coding_bootcamp.sql: the SQL scripts for the schema, sample data, triggers, and queries

6.2 Sources

- Hack Reactor - https://www.hackreactor.com/
- Cise Babatasi – Hack Reactor student